Instituto Tecnológico y de Estudios Superiores de Monterrey

TC2037 - Implementación de Métodos Computacionales

Ian Julián Estrada Castro – A01352823

Actividad Integradora 2 – Validación de entradas usando autómatas

26 de abril de 2024

Tabla de contenido

Expresiones *3*

Autómata *3*

Autómata Finito Determinista (AFD) 3

Programación funcional *4*

Scheme 4

Recursividad terminal 4

Actividad Integradora *5*

Movimientos (moves) 5

Estado de aceptación (accepting-state?) 6

Evaluar cadenas (evaluate-string) 6

Validaciones (validate) 6

Reflexión *7*

Expresiones

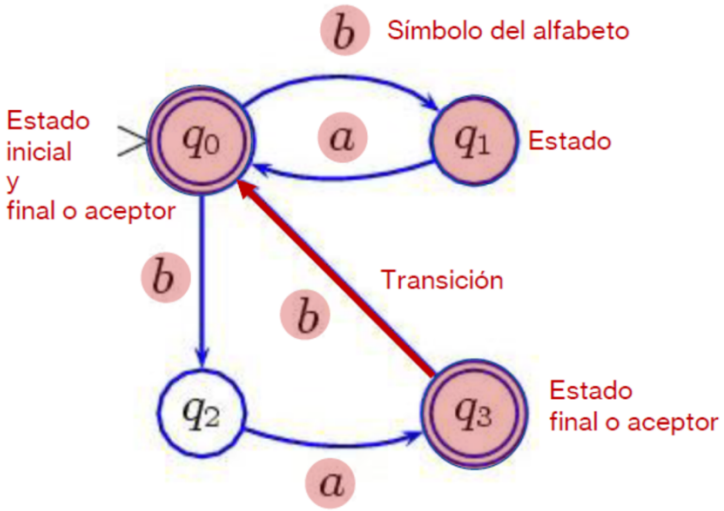
Secuencia de operandos unidos por operadores para dar un resultado. En una expresión los operadores pueden ser unarios o binarios los cuales tienen una prioridad para su ejecución.

Autómata

Modelo matemático funcional que sirve para describir un proceso de transformación (computacional) de una entrada en una salida. Se categorizan dependiendo del tipo de problemas que pueden resolver (capacidad o potencia computacional):

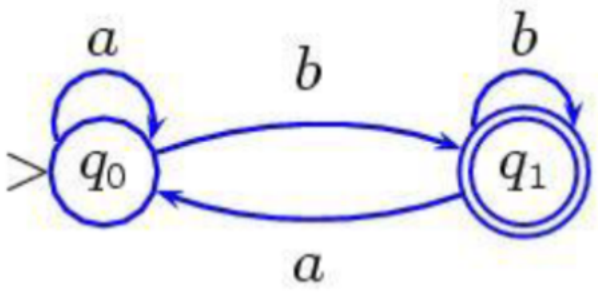
- Autómatas Finitos
- Autómatas de pila
- Máquinas de Turing

Un autómata finito puede describir con base en los estados del proceso y las transiciones entre los estados según los datos de entrada, hasta llegar a un estado de aceptación o rechazo. Los autómatas finitos pueden ser finitos deterministas (AFD) o no deterministas (AFN).



Autómata Finito Determinista (AFD)

En un autómata Determinista, para cada estado y símbolos de entrada, existe una transición única a otro estado. No hay ambigüedades para implementar.



Programación funcional

Es un paradigma de programación declarativa basado en el uso de verdaderas funciones matemáticas. Existen desde 1958 con la creación de ListP (List Processing)

- Todo se moldea con funciones: definiciones y llamadas, la secuencia es una composición de funciones.
- No se manejan variables.
- No se manejan asignaciones de valores.
- No hay iteraciones, sólo recursividad.
- Almacenan todo en listas encadenadas y con asociaciones dinámicas.
- Tratan a las funciones como a los datos: son argumentos, resultados y estructuras.

Scheme

Lenguaje de programación sencillo, minimalista, para aprender ágilmente el nuevo paradigma. Es un dialecto **Lisp** creado en 1975 por Steele y Sussman (MIT).

- Toda su sintaxis se reduce al formato de una lista que es una expresión-s:
    - Las **listas** se delimitan con paréntesis y puede tener cero o más elementos.
    - Una lista es una expresión-s cuyos elementos son a su vez una expresión-s.
- Tiene pocas reglas semánticas, pues es un lenguaje tipado débil (no hay declaraciones de tipos de datos).

Recursividad terminal

Estilo de programación recursiva en donde la solución al problema se obtiene al llegar al caso base (la “cola” del proceso). Se logra cuando la solución recursiva al problema no contiene operaciones adicionales a la llamada recursiva. No siempre corresponde al pensamiento natural recursivo y requerirá una conversión si se desea obtener sus ventajas.

Actividad Integradora

La actividad consistió en desarrollar un programa que permitiera evaluar cadenas de caracteres con base a un autómata finito determinista.
Usando racket (un lenguaje de programación de la familia de Scheme), se desarrolló la función validate que recibe como entrada la descripción formal del autómata y una lista de cadenas a evaluar. La función debía devolver una lista indicando si las diversas cadenas se aceptan o no.
El autómata en cuestión fue el siguiente:

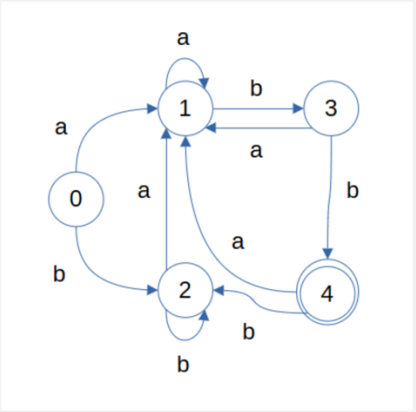
Definición formal de un AFD

Un Autómata Finito Determinístico (AFD) es un quinteto (K, Σ, δ, s_0, F) donde

- K es un conjunto finito no-vacío de ***estados***.
- Σ es el alfabeto de entrada (conjunto finito no-vacío de símbolos).
- $\delta : K \times \Sigma \rightarrow K$ es la ***función de transición***.
- $s_0 \in K$ es el ***estado inicial***.
- $F \subseteq K$ es el conjunto de ***estados finales***.

Determinístico: para cada combinación de $s \in K$ y $a \in \Sigma$, la función δ especifica uno y sólo un estado de transición.

5



La lista profunda para describir el autómata es: ((0 1 2 3 4) (a b) ((0 a 1) (0 b 2) (1 a 1) (1 b 3) (2 b 2) (2 a 1) (3 a 1) (3 b 4) (4 a 1) (4 b 2)) 0 (4))
La lista para describir las entradas es: ((a b a b a a b b) (a a a a a a b) (a b a b a b b) (a b))
La salida esperada para este autómata es: (#t #f #t #f).

Movimientos (moves)

Para iniciar se enfocó más en los movimientos posibles, para ello se creó la función moves, que a grandes rasgos responde las preguntas ¿dónde estoy? Y ¿A dónde puedo ir bajo cierto símbolo? Para ello se aplicó recursividad terminal, **move-helper** hace los movimientos con ayuda de **move**, que hace las comparaciones. Finalmente, **moves** llama a move-helper.

Estado de aceptación (accepting-state?)

Esta función es importante, ya que hace una comparación con el estado actual y la lista de estados de aceptación, responde a la pregunta ¿estoy en un estado de aceptación?

Evaluar cadenas (evaluate-string)

Como su nombre lo indica, evalua la cadena, esto lo hace con apoyo de **moves** y **accepting-state?**, conforme le la cadena, llama a moves para hacer los movimientos. Cuando la cadena termina, accepting-state? Determina si el estado en el que terminó es un estado de aceptación.

Validaciones (validate)

La función validate recibe una lista profunda con el autómata en su descripción formal y la lista de cadenas a evaluar. Lo que hace es extraer cada elemento de la lista profunda para después aplicar las funciones definidas. Lo que devuelve es una lista de **#t** y **#f** dependiendo de si la cadena es valida o no.

```
(define (validate automaton strings)
  (display "Estados: ") (display (car automaton)) (newline)
  (display "Alfabeto: ") (display (cadr automaton)) (newline)
  (display "Transiciones: ") (display (caddr automaton)) (newline)
  (display "Estado inicial: ") (display (cadddr automaton)) (newline)
  (display "Estados de aceptación: ") (display (cadr (cdddr automaton))) (newline)
  ;;; Extraer los componentes del autómata y las cadenas a evaluar
  (let* ((states (car automaton)) ; El primer elemento del automata
         (alphabet (cadr automaton)) ; El segundo elemento del automata
         (transitions (caddr automaton)) ; El tercer elemento del automata
         (initial-state (cadddr automaton)) ; El cuarto elemento del automata
         (accepting-states (cadr (cdddr automaton))) ; El quinto elemento del automata
         (states-valid? (validate-states-transitions states transitions)) ; Valida los estados
         (alphabet-valid? (validate-alphabet-transitions alphabet transitions)) ; Valida el alfabeto
         (result (map (lambda (string) ; Se usa map para aplicar evaluate-string a cada cadena de la lista de cadenas
                        (evaluate-string string transitions initial-state accepting-states))
                      strings)))
    (display "Estados válidos: ") (display states-valid?) (newline)
    (display "Alfabeto válido: ") (display alphabet-valid?) (newline)
    (display "Resultados de las cadenas: ") (display result) (newline)
    result)) ; Se verifica que las validaciones son verdaderas y devuelve el resultado
```

Para el automata de ejemplo, se obtuvo el siguiente resultado:

```
> (validate '((0 1 2 3 4) (a b) ((0 a 1) (0 b 2) (1 a 1) (1 b 3) (2 b 2) (2 a 1) (3 a 1) (3 b 4) (4 a 1) (4 b 2)) 0 (4))
            '((a b a b a a b b) (a a a a a a b) (a b a b a b b) (a b)))
Estados: (0 1 2 3 4)
Alfabeto: (a b)
Transiciones: ((0 a 1) (0 b 2) (1 a 1) (1 b 3) (2 b 2) (2 a 1) (3 a 1) (3 b 4) (4 a 1) (4 b 2))
Estado inicial: 0
Estados de aceptación: (4)
Estados válidos: #t
Alfabeto válido: #t
Resultados de las cadenas: (#t #f #t #f)
'(#t #f #t #f)
```

Reflexión

Hacer esta actividad no sólo requirió entender lo que es un autómata tanto AFN como AFD, y los algoritmos utilizados para leer su descripción formal, sino que también requirió de aprender un nuevo paradigma de programación, la programación funcional. Antes de este curso yo conocía de la programación funcional, pero nunca la use ni me dispuse a aprenderla. En este curso entendí mucho de la programación funcional y logré replicar problemas de otros paradigmas usando Scheme, algo que requirió un cambio mi forma de pensar, ya que se trató de otro paradigma. En un principio, la actividad se escuchó complicada, pero comenzar por pequeños fragmentos resultó de mucha utilidad. Inicié con los movimientos y después con las validaciones para al final, integrarlos en una función, apliqué conceptos del curso y al final logré hacer un programa de complejidad promedio de O(n+m+k), es probable que haya mejores formas de hacerlo. Esto me deja como área de oportunidad, mejorarlo para reducir la complejidad promedio, y para ello tendré que indagar más en el lenguaje y aprender más sobre la programación funcional.